

From MLPs to Trainable Models

Activations, objectives, and initialisation

Jonathon Hare and Antonia Marcu

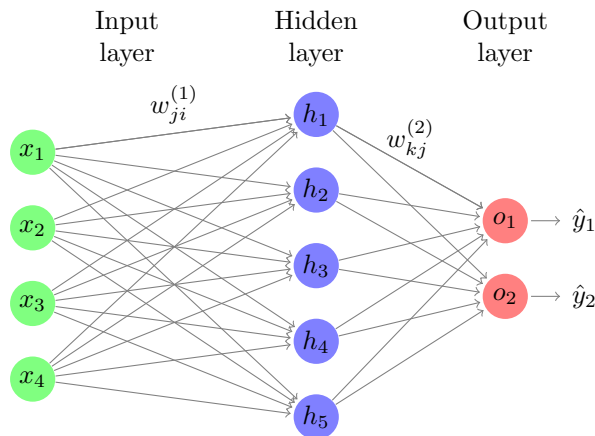
Topics

- Multilayer perceptrons as compositions of affine maps and nonlinearities
- Activations, logits, and loss functions
- The complete training pipeline
- Initialisation and the scale of signals and gradients
- Input scaling, weight initialisation, and internal normalisation

How Supervised Learning Typically Works

- Start by choosing a model-class: $\hat{y} = f(\mathbf{x}; \mathbf{W})$ where the model-class f is a way of using some numerical parameters, $\mathbf{W}$, to map each input vector $\mathbf{x}$ to a predicted output $\hat{y}$.
- *Learning* means adjusting the parameters to reduce the discrepancy between the true target output y on each training case and the output $\hat{y}$, predicted by the model.

Let's look at a Multilayer Perceptron (without biases)...



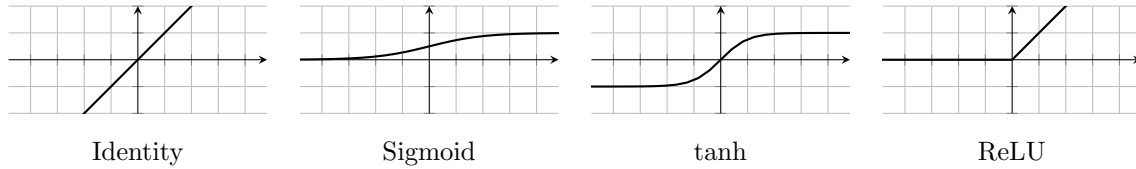
Without loss of generality, we can write the above as:

$$\hat{\mathbf{y}} = g(f(\mathbf{x}; \mathbf{W}^{(1)}); \mathbf{W}^{(2)}) = g(\mathbf{W}^{(2)} f(\mathbf{W}^{(1)} \mathbf{x}))$$

where f and g are activation functions.

Common Activation Functions

- Identity
- Sigmoid (aka Logistic)
- Hyperbolic Tangent (tanh)
- Rectified Linear Unit (ReLU) (aka Threshold Linear)



Final layer activations

$$\hat{\mathbf{y}} = g(\mathbf{W}^{(2)} f(\mathbf{W}^{(1)} \mathbf{x}))$$

- What form should the final layer function g take?
- It depends on the task (and on the chosen loss function)...
 - For regression it is typically linear (e.g. identity), but you might choose others if you wanted to clamp the range of the network.
 - For binary classification (MLP has a single output), one would choose Sigmoid
 - For multilabel classification, typically one would choose Sigmoid
 - For multiclass classification, typically you would use the Softmax function

Softmax

The softmax is an activation function used at the output layer of a neural network that forces the outputs to sum to 1 so that they can represent a probability distribution across a discrete mutually exclusive alternatives.

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad \forall i = 1, 2, \dots, K$$

- The outputs are positive and sum to 1.
- The denominator contains every logit, so every output depends on every input.
- Let $\mathbf{s} = \text{softmax}(\mathbf{z})$. The entries of its Jacobian are:

$$\frac{\partial s_i}{\partial z_j} = s_i(\delta_{ij} - s_j).$$

Here δ_{ij} is the Kronecker delta: it is 1 if $i = j$, and 0 otherwise.

Softmax as a relaxation of one-hot argmax

- A multiclass decision selects the largest logit:

$$c = \arg \max_k z_k.$$

- The equivalent one-hot vector is discontinuous: a small change in the logits can switch the selected class.
- Softmax replaces the hard one-hot vector with a continuous, differentiable distribution:

$$\begin{aligned} \mathbf{z} &= \begin{bmatrix} 1.1 & 4.0 & -0.1 & 2.3 \end{bmatrix} \\ \text{onehot}(\arg \max_k z_k) &= \begin{bmatrix} 0 & 1 & 0 & 0 \end{bmatrix} \\ \text{softmax}(\mathbf{z}) &= \begin{bmatrix} 0.044 & 0.797 & 0.013 & 0.146 \end{bmatrix} \end{aligned}$$

- The largest component stays associated with the predicted class, but the output retains information about the alternatives.

Temperature controls the relaxation

$$\text{softmax}(\mathbf{z}/T)_i = \frac{e^{z_i/T}}{\sum_{j=1}^K e^{z_j/T}}, \quad T > 0.$$

- Lower temperatures sharpen the distribution:

$$\begin{aligned} T = 1.0 : & \begin{bmatrix} 0.044 & 0.797 & 0.013 & 0.146 \end{bmatrix} \\ T = 0.6 : & \begin{bmatrix} 0.008 & 0.937 & 0.001 & 0.055 \end{bmatrix} \\ T = 0.2 : & \begin{bmatrix} 5.0 \times 10^{-7} & 0.9998 & 1.3 \times 10^{-9} & 0.0002 \end{bmatrix} \end{aligned}$$

- If the largest logit is unique, then as $T \rightarrow 0^+$:

$$\text{softmax}(\mathbf{z}/T) \longrightarrow \text{onehot}(\arg \max_k z_k).$$

- Higher temperatures flatten the distribution. As $T \rightarrow \infty$, it approaches the uniform distribution.

Ok, so let's talk loss functions

- The choice of loss function depends on:
 1. the task (e.g. classification/regression/something else)
 2. the activation function of the last layer
 - For numerical reasons (see Log-Sum-Exp in a few slides)
 - the activation is often computed directly within the loss rather than being part of the model
- Some classification losses require *raw outputs* (e.g. a linear layer) of the network as their input
 - These are often called *unnormalised log probabilities* or *logits*
 - An example would be hinge-loss used to create a Support Vector Machine for binary classification that maximises the margin — e.g.: $\ell_{\text{hinge}}(\hat{y}, y) = \max(0, 1 - y \cdot \hat{y})$ with a true label, $y \in \{-1, 1\}$
- There are many different loss functions we might encounter (MSE, Cross-Entropy, KL-Divergence, huber, L1 (MAE), CTC, Triplet, ...) for different tasks.

The Loss Function (measure of discrepancy)

Recall from Foundations of Machine Learning:

- Mean Squared Error (MSE) loss for a single data point (here assumed to be a vector, but equally applicable to a scalar)
- is given by
- $$\ell_{MSE}(\hat{\mathbf{y}}, \mathbf{y}) = \sum_i (\hat{y}_i - y_i)^2 = (\hat{\mathbf{y}} - \mathbf{y})^\top (\hat{\mathbf{y}} - \mathbf{y})$$
- We often multiply this by a constant factor of $\frac{1}{2}$ — can anyone guess/remember why?
- $\ell_{MSE}(\hat{\mathbf{y}}, \mathbf{y})$ is the predominant choice for regression problems with linear activation in the last layer
- For a classification problem with Softmax or Sigmoidal (or really anything non-linear)
 - activations MSE can cause slow learning, especially if the predictions are very far off the targets
 - Gradients of ℓ_{MSE} are proportional to the difference in target and predicted value, multiplied by the gradient of the activation function¹
 - **The Cross-Entropy loss function is generally a better choice in this case**

Binary Cross-Entropy

For the binary classification case:

$$\ell_{BCE}(\hat{y}, y) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$

- The cross-entropy loss function is non-negative, $\ell_{BCE} > 0$
- $\ell_{BCE} \approx 0$ when the prediction and targets are equal (i.e. $y = 0$ and $\hat{y} = 0$ or when $y = 1$ and $\hat{y} = 1$)

¹<http://neuralnetworksanddeeplearning.com/chap3.html>

- With Sigmoidal final layer, $\frac{\partial \ell_{BCE}}{\partial \mathbf{W}_i^{(2)}}$ is proportional to just the error in the output ($\hat{y} - y$) and therefore, the larger the error, the faster the network will learn!
- Note that the BCE is the negative log likelihood of the Bernoulli Distribution

Binary Cross-Entropy — Intuition

- The cross-entropy can be thought of as a **measure of surprise**.
- Given some input x_i , we can think of $\hat{y}_i$ as the estimated probability that x_i belongs to class 1, and $1 - \hat{y}_i$ is the estimated probability that it belongs to class 0.
- Note the extreme case of infinite cross-entropy, if your model believes that a class has 0 probability of occurrence, and yet the class appears in the data, the ‘surprise’ of your model will be infinitely great.

Binary Cross-Entropy for multiple labels

In the case of multi-label classification with a network with multiple sigmoidal outputs you just sum the BCE over the outputs:

$$\ell_{BCE} = - \sum_{k=1}^K [y_k \log(\hat{y}_k) + (1 - y_k) \log(1 - \hat{y}_k)]$$

where K is the number of classes of the classification problem, $\hat{y} \in \mathbb{R}^K$.

Numerical Stability: The Log-Sum-Exp trick

$$\ell_{BCE}(\hat{y}, y) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$

- Consider what might happen early in training when the model might confidently predict a positive example as negative
 - $\hat{y} = \sigma(z) \approx 0 \implies z \ll 0$
 - if $\hat{y}$ is small enough, it will become 0 due to limited precision of floating-point representations
 - but then $\log(\hat{y}) = -\infty$, and everything will break!
- To tackle this problem implementations usually combine the sigmoid computation and BCE into a single loss function that you would apply to a network with linear outputs (e.g. `BCEWithLogitsLoss`).
- Internally, a trick called ‘log-sum-exp’ is used to *shift* the centre of an exponential sum so that only numerical underflow can potentially happen, rather than overflow²
- .
 - Ultimately this means you’ll always get a numerically reasonable result (and will avoid NaNs and Infs originating from this point).

Multiclass classification with Softmax Outputs

- Softmax can be thought of making the K outputs of the network mimic a probability distribution.
- The target label y could also be represented as a distribution with a single 1 and zeros everywhere else.
 - e.g. they are “one-hot encoded”.
- In such a case, the obvious loss function is the *negative log likelihood* of the Categorical distribution (aka Multinoulli, Generalised Bernoulli, Multinomial with one sample)³: $\ell_{NLL} = - \sum_{k=1}^K y_k \log \hat{y}_k$
 - Note that in practice as y_k is zero for all but one class you don’t actually do this summation, and if y is an integer class index you can write $\ell_{NLL} = -\log \hat{y}_y$.
- Analogously to what we saw for BCE,

Log-Sum-Exp can be used for better numerical stability. PyTorch combines LogSoftmax with NLL in one loss and calls this “Categorical Cross-Entropy” (so you would use this with a **linear output layer**)

²<https://www.xarg.org/2016/06/the-log-sum-exp-trick-in-machine-learning/>

³Note: Keras calls this function ‘Categorical Cross-Entropy’; you would need to have a Softmax output layer to use this

The complete training pipeline

Why zero initialisation fails

The scale of the initial weights matters

Variance propagation through a layer

Xavier and Glorot initialisation

He and Kaiming initialisation

Bias initialisation and PyTorch defaults

Input scaling, initialisation, and internal normalisation